Modeling Human Activity States Using Hidden Markov Models

Henry Christian Parfait Uhiriwe

Machine Learning Techniques 2

African Leadership University

1 November 2026

GitHub Repository: Link

1. Background and Motivation

My use case is an **active-break coach for students**: a phone app that notices when a student has been stationary for too long and confirms whether they actually took a movement break. As a student, I regularly spend long evening sessions at a desk with my phone next to my laptop, and I rarely notice how long I go without moving. Prolonged sitting is widely linked to poorer physical and mental health, and short breaks of light activity are a simple, recommended countermeasure. The four activities in this project map directly onto that scenario: *still* (phone lying on the desk while I study), *standing* and *walking* (light activity), and *jumping* (a short exercise break such as jumping jacks). The coach cannot ask the student what they are doing every second; it only sees noisy accelerometer and gyroscope readings, while the true activity stays hidden. Activities also follow each other over time, so a still period is usually followed by standing up, not by an instant jump. A Hidden Markov Model fits this problem naturally: it infers the hidden activity from noisy observations and uses transition probabilities to model how activities follow one another.

2. Data Collection and Pre-processing

I recorded 52 labelled training clips (13 per activity) and 2 continuous test sessions with one phone, all resampled to a common 100 Hz.

Device and sampling rate. This was a solo project, so all data comes from one device.

Member	Phone	App	Sensors	Configured rate	Measured rate
Henry	Huawei P20 (EML-L09), Android 9	Sensor Logger 1.68	Accelerometer (linear), Gyroscope, Gravity	100 Hz	98.0 Hz (max gap 19 ms)

I also recorded the Gravity stream because Sensor Logger's accelerometer has gravity removed; gravity tells the model how the phone is oriented.

Recording protocol. For standing, walking, and jumping, I held the phone upright in portrait at belt level, screen facing me, always in the same hand. For still, the phone lay flat on a table, screen up. Each clip ran about 15 s; I tapped Start, waited a moment, performed the activity, then tapped Stop. Do Not Disturb was on so that notification vibrations could not create false spikes. Clips were named `activity_NN` (e.g., `walking_07`) directly in the app.

Video evidence. Each activity was filmed with my laptop webcam from about 3 m away, showing my full body and face while the phone recorded.

Activity	Video
Standing	Link
Walking	Link
Jumping	Link
Still	Link

Cleaning and harmonization. Each raw recording was processed in four steps:

1. **Trim:** the first 2 s and last 1 s were removed. A pilot recording showed that tapping Start, getting into the activity, and the sensor filters settling all distort the first ≈1.5 s.
2. **Crop:** the central 10 s of each clip was kept, so every clip has the same length.
3. **Resample:** Android delivered samples at irregular intervals (≈98 Hz on average, gaps up to 19 ms). Linear interpolation placed all three sensors on one shared, evenly spaced 100 Hz time grid, which also aligns sensors that started a fraction of a second apart.
4. **Save:** each clip became one tidy CSV (`timestamp_ns`, `seconds`, 9 sensor columns, `activity`).

The result is 13 files × 10 s = **130 s per activity**, above the 90 s minimum.

Windowing. Each clip was cut into **2 s windows (200 samples) with 50% overlap (1 s step)**, giving 9 windows per clip and 117 per activity (468 in total). Two seconds covers at least two full cycles of the slowest rhythmic activity (walking, ≈1–2 Hz). At 100 Hz, it gives an FFT resolution of 0.5 Hz, refined to ≈0.2 Hz by zero-padding to 512 points. The 1 s step produces one observation per second for the HMM.

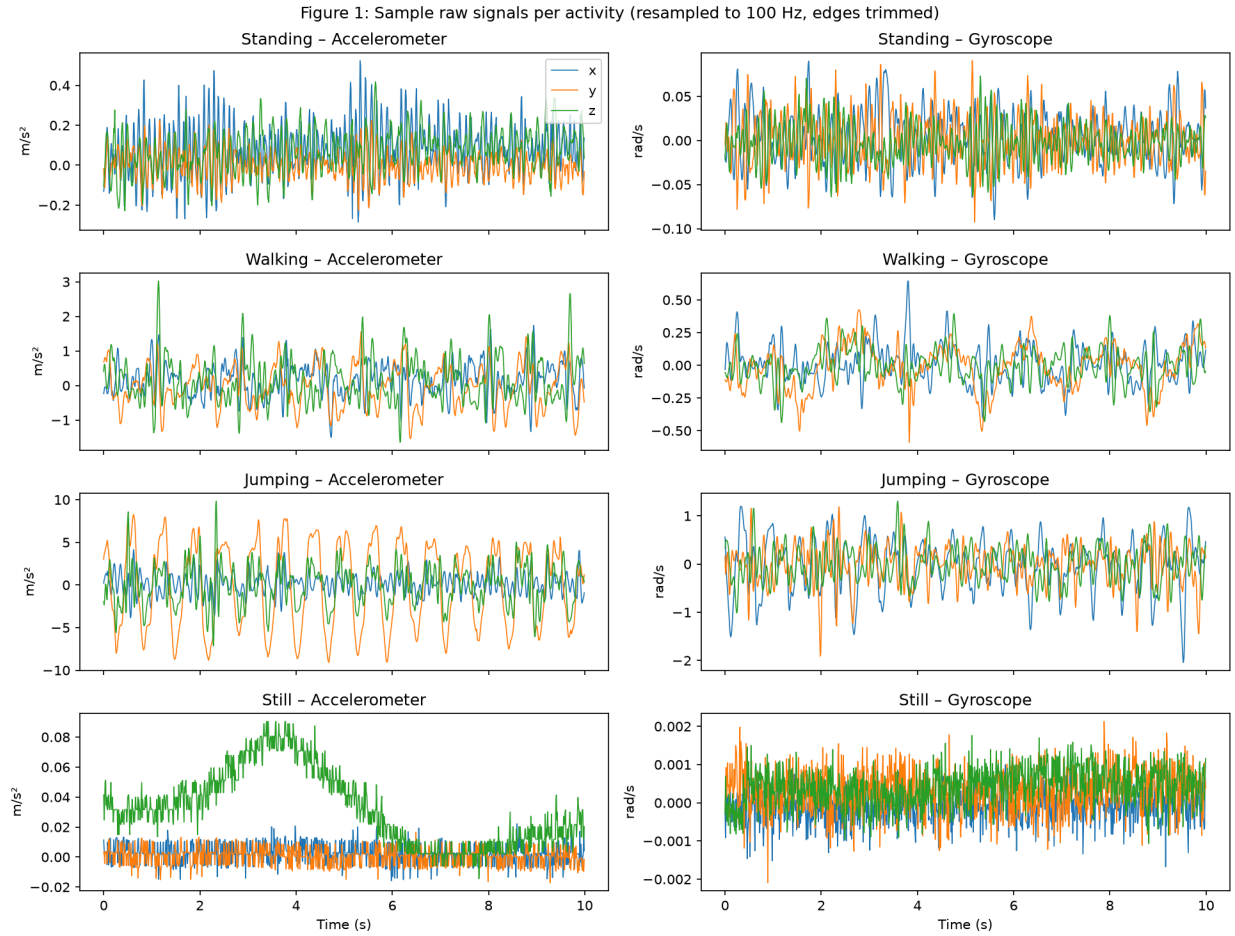


Figure 1: One sample clip per activity after cleaning and resampling (accelerometer left, gyroscope right). Each panel has its own y-axis scale: still moves by hundredths of m/s², jumping by ±10 m/s².

3. Feature Extraction and Normalization

Each window is summarized by 12 features (8 time-domain, 4 frequency-domain from the FFT); together they form the HMM's observation sequence. Magnitudes ($\sqrt{(x^2+y^2+z^2)}$) are used where possible because they do not depend on exactly how the phone is tilted in the hand.

Feature	Domain	What it captures and why it helps
acc_mag_mean	Time	Average movement intensity; orders the activities: still < standing < walking < jumping
acc_mag_std	Time	How much intensity varies; high for the impacts of jumping, near zero when still
acc_sma	Time	Signal magnitude area, sum of absolute acceleration on all axes; a standard activity-level measure

Feature	Domain	What it captures and why it helps
gyro_mag_mean	Time	Average rotation; walking and jumping swing the hand, still has none
gyro_mag_std	Time	Variability of rotation; separates steady standing sway from rhythmic movement
acc_corr_yz	Time	Correlation between the y and z axes; jumping moves both axes together (≈0.74)
grav_y_mean	Time	Gravity on the y-axis; ≈9.4 m/s² when the phone is upright
grav_z_mean	Time	Gravity on the z-axis; ≈9.8 m/s² when the phone lies flat, isolating still
acc_dom_freq	Frequency	Peak of the acceleration spectrum; the rhythm of the movement (walking ≈1.4 Hz, jumping ≈3.6 Hz)
acc_band_ratio	Frequency	Share of spectral energy in the 0.5–5 Hz human-movement band; high for walking and jumping
acc_spec_entropy	Frequency	Flatness of the spectrum; low for rhythmic activities, high for noise-like standing and still
gyro_dom_freq	Frequency	Peak of the rotation spectrum; a second, independent estimate of rhythm

Before each FFT, the window's mean is removed, and a Hann window is applied to reduce spectral leakage. Jumping's dominant frequency (≈3.6 Hz) is about twice the jump rate, because each jump produces two acceleration peaks: take-off and landing.

Normalization. The intensity features span three orders of magnitude, from ≈0.02 m/s² (still) to ≈5.7 m/s² (jumping). The five intensity features are therefore **log10-transformed** first. This spreads the low-motion classes apart and makes each class's distribution closer to Gaussian, matching the emission model. All 12 features are then **Z-score normalized** (mean 0, standard deviation 1) so that no feature dominates the likelihood because of its units. The mean and standard deviation are fitted on the training windows only and reused unchanged on the test data, so no information leaks from the unseen set.

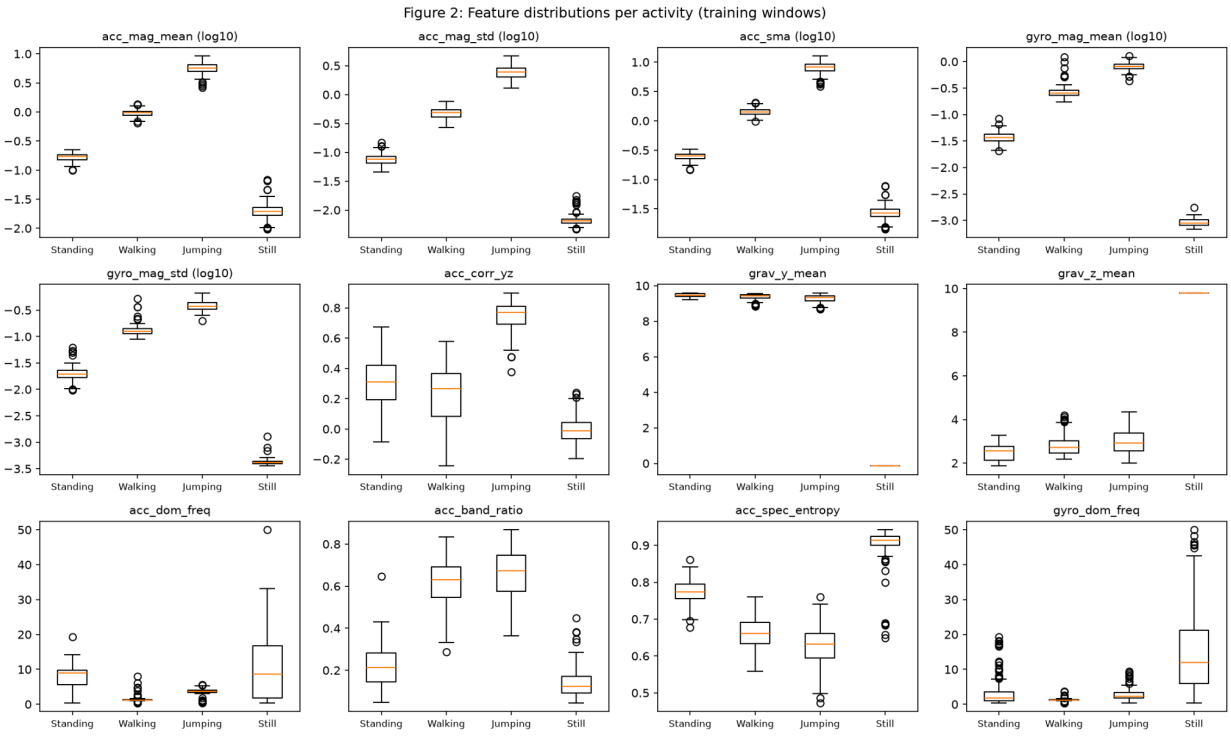


Figure 2: Distribution of each feature per activity over the 468 training windows. The four intensity features (top row) separate all activities with no overlap; `grav_z_mean` isolates still. The two dominant-frequency features are noisy for standing and still, which have no rhythm, but they help separate walking from jumping.

4. HMM Setup and Implementation

The model is a 4-state HMM with Gaussian emissions, implemented from scratch in NumPy (forward-backward, Baum-Welch, and Viterbi), with all computations in log-space.

Element	In this project
Hidden states (Z)	4 activities: standing, walking, jumping, still
Observations (X)	One 12-dimensional normalized feature vector per 2 s window, one per second
Transition probabilities (A)	4×4 matrix, P(activity at t+1 | activity at t)
Emission probabilities (B)	One Gaussian per state over the 12 features, diagonal covariance
Initial probabilities (π)	P(activity of the first window)

Algorithm choice. Three algorithms work together, each tied to a part of the evaluation:

- **Baum-Welch (EM)** trains the model, re-estimating π, A, and the Gaussian parameters to maximize the likelihood of the training sequences. Training stops when the log-likelihood improves by less than $\varepsilon = 10^{-4}$, with a safety cap of 200 iterations.
- **Forward-backward** supplies the state posteriors that Baum-Welch needs. The forward pass also gives the sequence log-likelihood used as the convergence criterion.
- **Viterbi** returns the single most likely activity sequence for a recording. These predicted labels are compared with the video-based labels to produce the confusion matrix, sensitivity, specificity, and accuracy.

Design decisions.

- *Log-space with log-sum-exp:* multiplying hundreds of small probabilities underflows to zero in ordinary floating point.
- *Diagonal covariance:* a full covariance needs 78 parameters per state with only 117 windows per state, and the intensity features are strongly correlated, which makes full covariances nearly singular.
- *Variance floor of 0.01:* some features barely vary within a class (gravity direction when the phone lies still). Without a floor, the Gaussian collapses, and a slightly tilted table on another day would receive near-zero likelihood.
- *Probability floor of 10^{-6} on π and A:* no transition is ever impossible, so the model can still follow a change it never saw in training.

Training sequences. Every training clip contains only one activity, so on its own it contains no transitions to learn from. I therefore built 13 training sessions, each joining one clip of every activity in a random order (fixed seed 42). Each session has 4 bouts of 9 windows (36 windows). π, A, and the Gaussians were first estimated from the labelled windows, which anchor each hidden state to a known activity. Baum-Welch then refined all parameters without labels.

Verification. Before using real data, I tested the implementation on synthetic data from a known 3-state HMM, starting from labels that were 30% randomly wrong. The log-likelihood increased at every iteration and converged after 9 iterations. The learned transition matrix was close to the true one (first row 0.89 / 0.04 / 0.07 vs. 0.90 / 0.05 / 0.05), and Viterbi recovered 97.3% of the true states.

On the real data, Baum-Welch **converged after 1 iteration** with a training accuracy of 100% both before and after. This is expected: the activities are so well separated (Figure 2) that the supervised starting point is already at the likelihood maximum, so EM confirms it rather than changing it. The synthetic test above shows that the algorithm does real work when the starting point is poor.

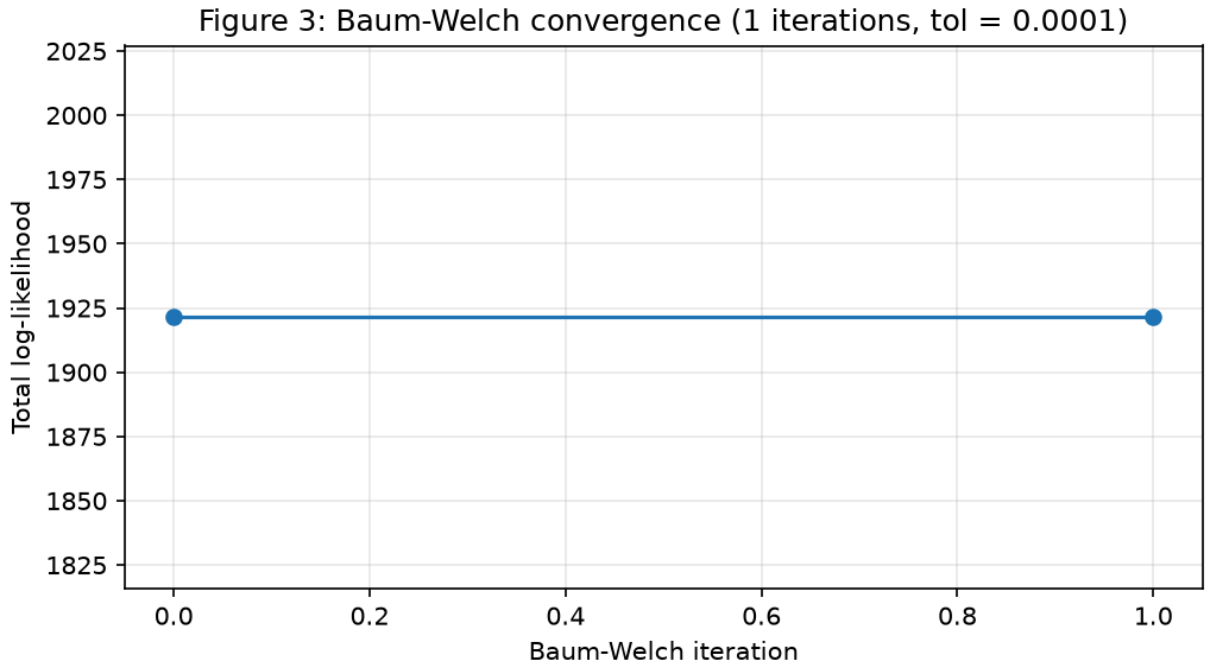


Figure 3: Total log-likelihood of the training sessions per Baum-Welch iteration. The gain after the first iteration was below $\varepsilon = 10^{-4}$.

5. Results and Interpretation

On two unseen test sessions, the model reached **94.4% accuracy (119 of 126 windows)**, and every error fell within about 2 s of an activity change.

Learned parameters. The transition matrix (Figure 4) has self-transition probabilities of 0.90–0.92, meaning an activity persists for about 11 s on average (1/(1−0.91)). Off-diagonal values are small (0.018–0.053), and jumping → still is effectively zero because that order never occurred in the training sessions. The emission means (Figure 5) step up evenly across intensity features, from −1.4 (still) through −0.4 (standing) and +0.5 (walking) to +1.3 (jumping). Still stands out on `grav_z_mean` (+1.7) and spectral entropy (+1.4). In Figure 6, the Gaussians for the intensity features barely overlap, which explains the perfect training accuracy.

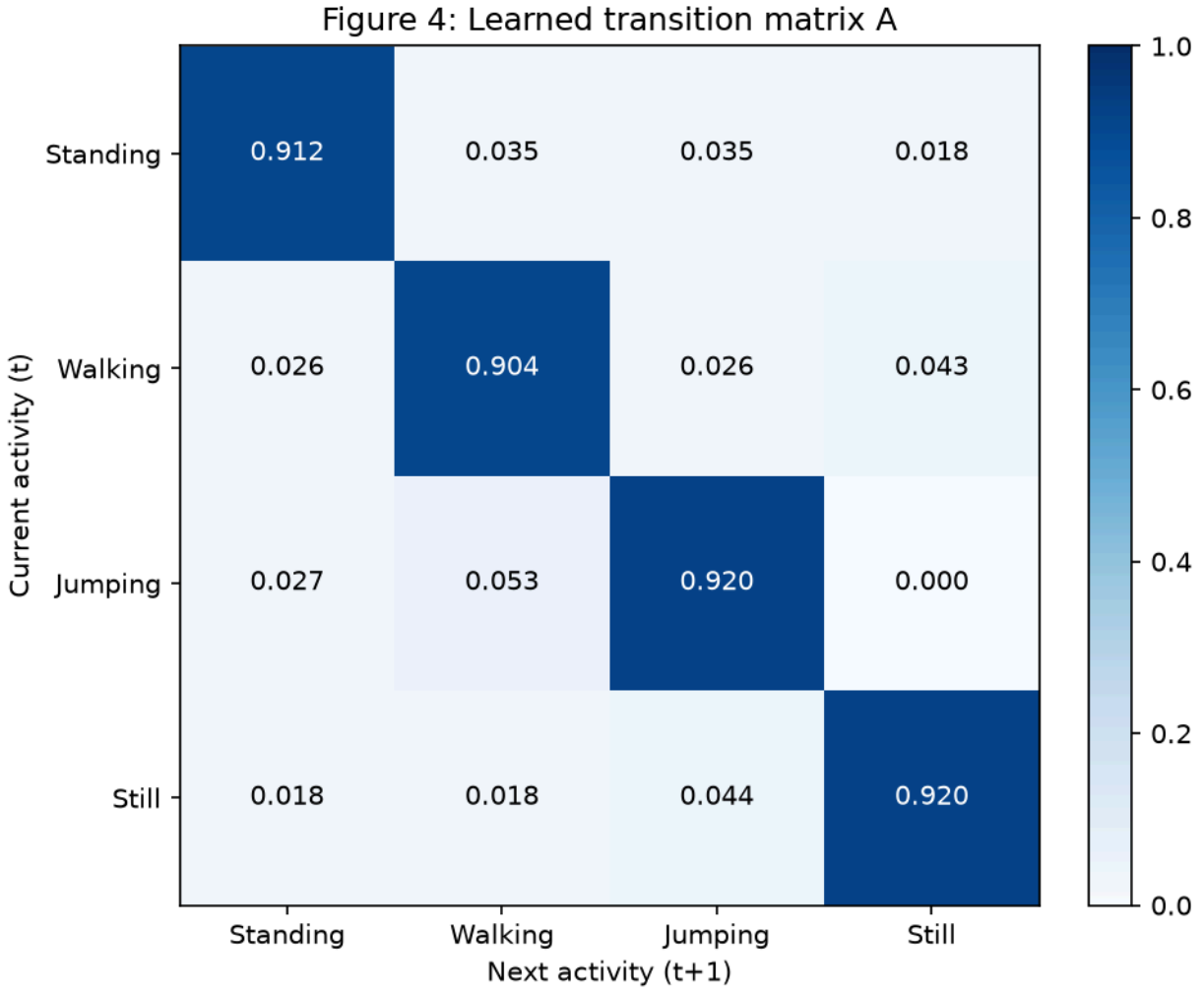


Figure 4: Learned transition matrix A (rows: current activity, columns: next activity).

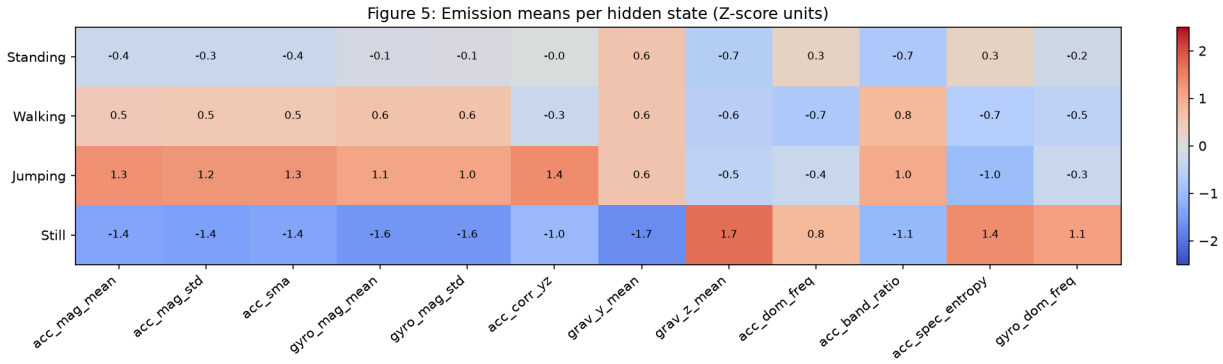


Figure 5: Emission means per hidden state for all 12 features, in Z-score units.

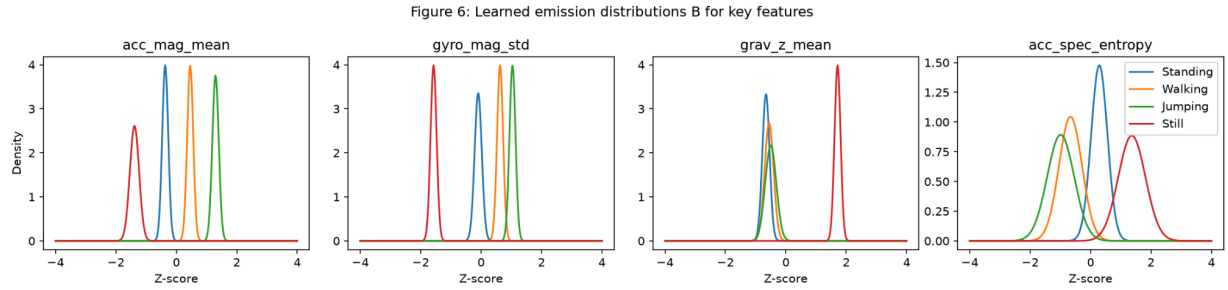


Figure 6: Learned emission Gaussians for four key features.

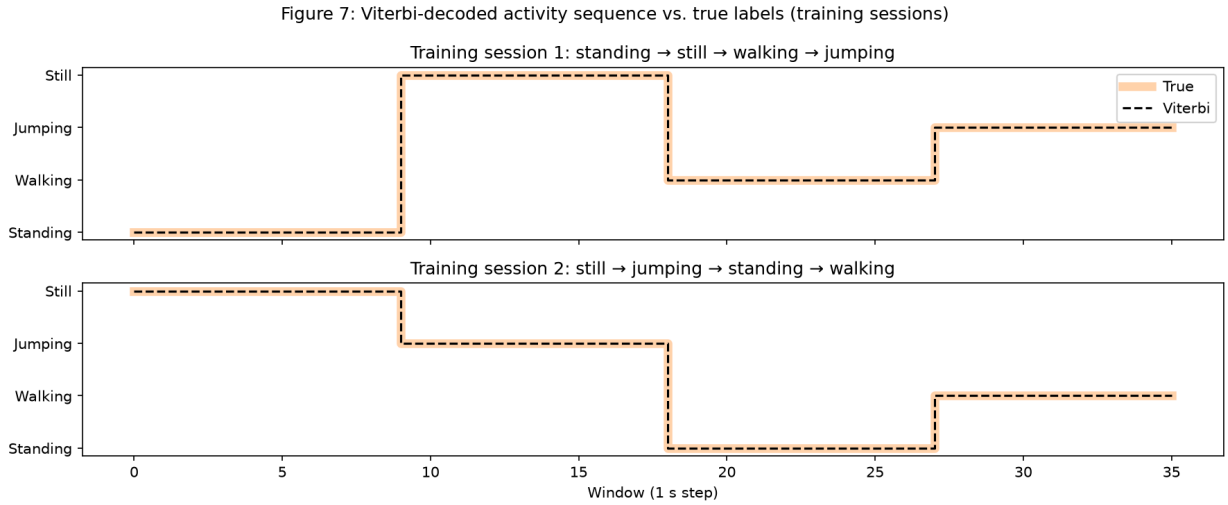


Figure 7: Viterbi-decoded sequences for two training sessions; the prediction matches the true labels exactly (100% training accuracy).

How the unseen data was obtained. Later on the same day as the training clips, in the same room, I recorded two new continuous sessions, `test_01` (64.3 s) and `test_02` (65.9 s), that were never used in training. Unlike the training clips, each session contains real transitions: walking → standing → jumping → still, performed without stopping. Both sessions were filmed, and I used the video to mark when each activity started and ended. Each window was labelled with the activity at its centre. The test data went through the same pipeline and was normalized with the training mean and standard deviation.

Evaluation metrics (both test sessions combined, 126 windows).

State (Activity)	Number of samples (windows)	Sensitivity	Specificity	Per-class accuracy (TP+TN)/N	Overall accuracy
Standing	21	0.857	1.000	0.976	0.944
Walking	39	1.000	0.931	0.952	0.944
Jumping	38	0.947	0.989	0.976	0.944

State (Activity)	Number of samples (windows)	Sensitivity	Specificity	Per-class accuracy (TP+TN)/N	Overall accuracy
Still	28	0.929	1.000	0.984	0.944

Sensitivity is TP/(TP+FN), the share of each activity's windows that were recognized. Specificity is TN/(TN+FP), the share of other activities' windows that were correctly not given this label. Per session, accuracy was 95.2% for `test_01` and 93.8% for `test_02`.

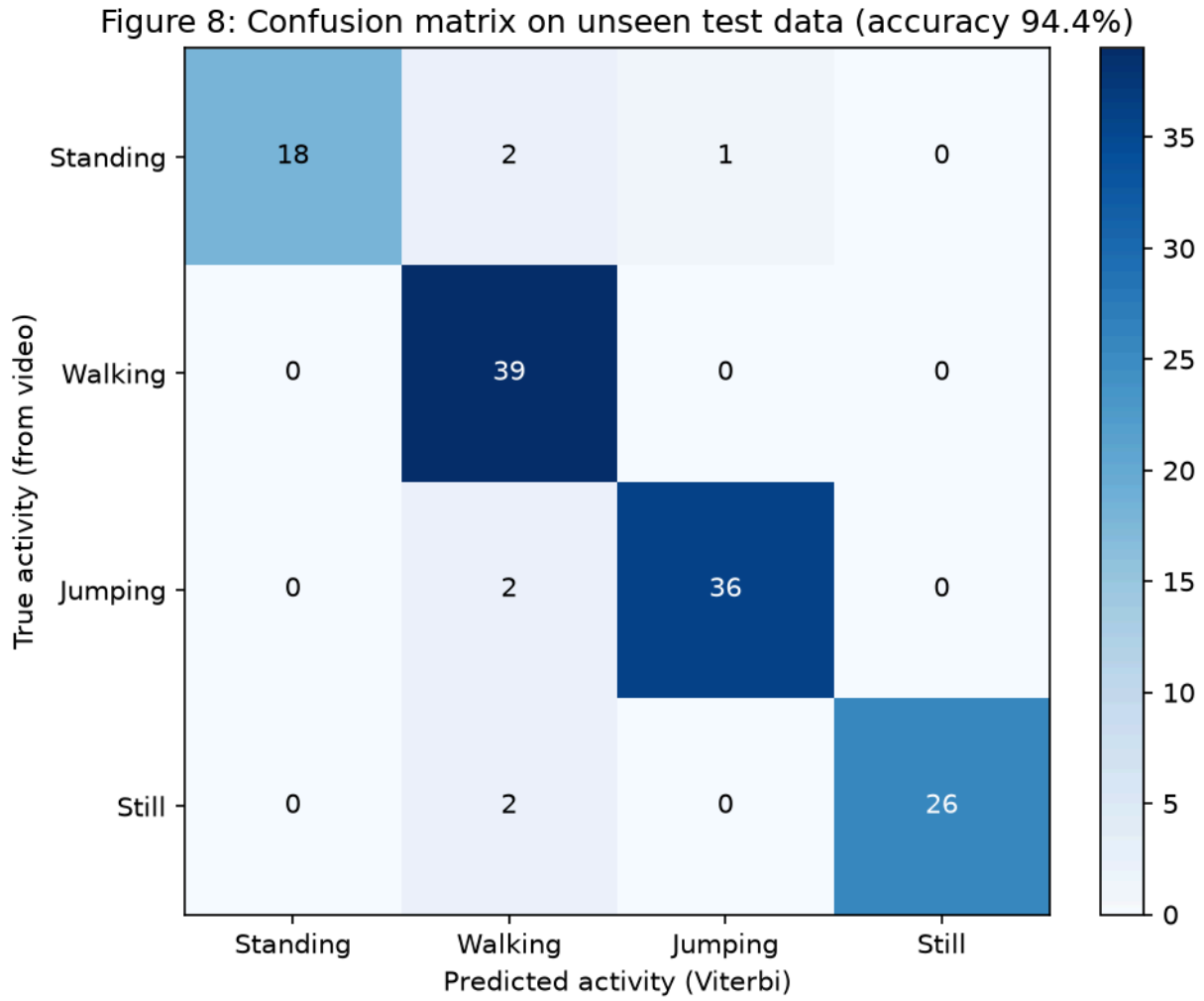


Figure 8: Confusion matrix on the unseen test sessions (rows: true activity from video, columns: Viterbi prediction).

Figure 9: Viterbi-decoded activity sequences on unseen test sessions

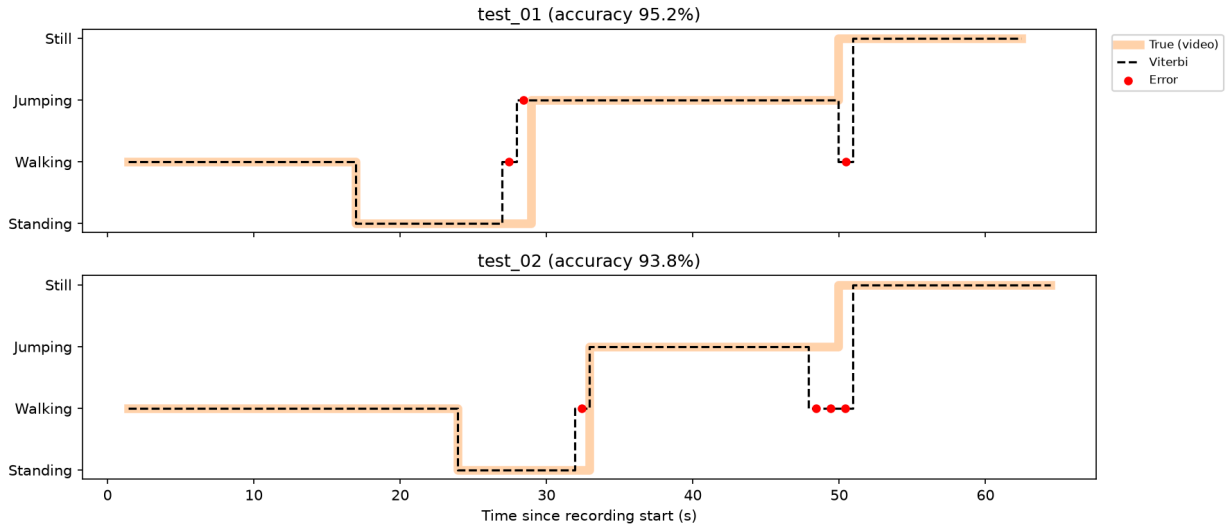


Figure 9: Viterbi-decoded sequences (dashed) against the video labels (orange) for both test sessions; red dots mark errors.

Interpretation. Figure 9 shows the model following both sessions closely, with no errors inside any stable activity. All 7 errors sit at transitions, where a 2 s window contains parts of two activities. Six of the seven errors were predicted as walking, which gives walking perfect sensitivity (1.000) but the lowest specificity (0.931). A mixed window, such as half jumping and half still, has medium intensity, and medium intensity is exactly where walking's Gaussian lies. The model therefore **generalizes well to the new session**, with the caveat that it was tested on the same person, phone, and place.

6. Discussion

Easiest and hardest activities. Walking was the easiest activity (39/39 windows, sensitivity 1.000), followed by jumping (0.947) and still (0.929), whose only errors were at transitions. Standing was the hardest (0.857): all three of its errors came in the last seconds before jumping, when I was already preparing to jump. Inside stable segments, every activity was recognized, because the intensity features form a clean ladder with no overlap (Figure 2). Without the orientation features, standing versus still would have been the hardest pair: both involve almost no motion. Holding the phone upright for standing and laying it flat for still put gravity on different axes, and `grav_z_mean` alone separates the two.

Do the transition probabilities reflect realistic behaviour? Only partly, and the reason is instructive. The diagonal (≈0.91) implies bouts of about 11 s, which matches the length of my clips rather than real behaviour. A real study session keeps the phone still for many minutes, which at one window per second means a self-transition probability above 0.999. The off-diagonal values come from the random order of the training sessions, so they treat still → jumping as likely as still → standing. In reality, a student almost always stands up before exercising. The test sessions exposed this limit: jumping → still never occurred in training, so Viterbi routed through walking (0.053, then 0.043) instead. Physically, ending a jump series and putting the phone on the table involves a brief moment of movement that the video labels

compressed into an instant change, so part of this "error" may be real in-between movement. Viterbi and per-window classification gave identical accuracy, because the emission evidence was decisive at every step. With noisier features, a phone in a pocket or a different person, the transition matrix would matter more, and realistic transitions would need long, natural recordings.

Effect of sensor noise and sampling rate. The phone delivered ≈98 Hz with irregular gaps of up to 19 ms; resampling to an even 100 Hz grid fixed this and aligned the three sensors. A rate of 100 Hz is far more than needed, since human movement is below about 10 Hz, so the rate never limited accuracy. Noise mattered mainly for the low-motion classes. The "still" accelerometer drifted by up to 0.08 m/s² while the phone lay untouched (Figure 1), because Android computes linear acceleration by subtracting an *estimated* gravity that takes seconds to settle. The first ≈1.5 s of each recording also showed filter warm-up, which is why 2 s were trimmed. Noise also makes the dominant frequency meaningless for standing and still, which have no rhythm. Finally, the gravity features of still vary so little that the variance floor was essential to keep the Gaussians from collapsing.

Improvements. Four changes would make the model more realistic and robust:

1. **More varied data:** recordings from other people, rooms, days, and phone positions (pocket, bag) to test whether the model generalizes beyond one person and one grip.
2. **Natural, transition-rich sessions:** long recordings of real study sessions would let Baum-Welch learn true bout lengths and realistic transitions.
3. **Better handling of transitions:** a shorter step (e.g., 0.5 s) or an explicit "transition" state would reduce the boundary errors that caused all 7 mistakes.
4. **Additional sensors and activities:** adding *sitting* (the real sedentary state for the use case) and a barometer to detect stairs; with more data, full-covariance or mixture emissions could model correlated features properly.

7. Conclusion

A 4-state Gaussian HMM built from scratch in NumPy recognized standing, walking, jumping, and still with **94.4% accuracy on unseen sessions**, with every error within about 2 s of an activity change. The project showed that well-chosen features carry most of the work: movement intensity orders the activities, and phone orientation separates the two motionless ones. The HMM adds a principled model of time, verified on synthetic data, but its transition matrix is only as realistic as the sequences it learns from. For the active-break coach, the next step is recording long, natural study sessions, including sitting, so that the model learns how students actually move between rest and activity.